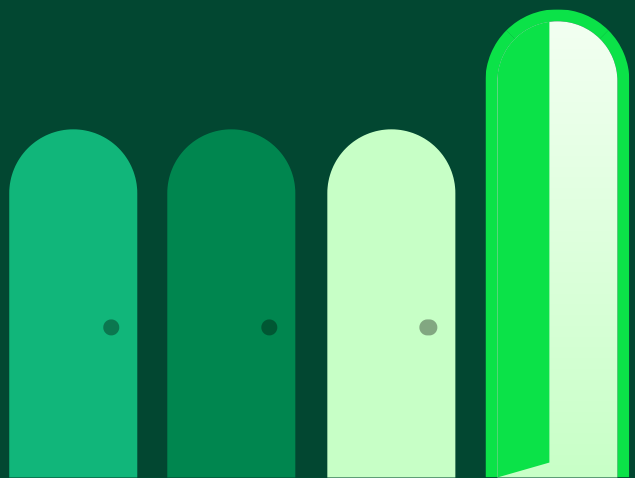
GitHub Copilot skill · Payments

One-Way Door

A GitHub Copilot skill that checks every change for a way back, before it is merged.

Know before you merge ›

One way only ›



In one paragraph

Rollback is the safety net for every release, yet nothing checks that it works. A change is a **one-way door** when it leaves behind state the previous release can't handle: a migrated table, new-format messages, rule data in another repo, a payment already sent to SWIFT. **One-Way Door** is a GitHub Copilot skill that reviews each pull request for exactly this. It says whether rollback is safe and for how long, finds who else is affected, and proposes a safe sequence of smaller changes.

Team	[Team name]
Domain	Payments: SWIFT, GTX gateway, payment hub
Runs in	VS Code agent mode · Copilot CLI · Copilot cloud agent
Date	12 October 2026

At a glance (measured)

4 pass · 1 fail
rollback tests, exactly as the skill predicted

61
automated tests across scripts, hook and plugin

0 fakes
every rollback rebuilds the old code from git

01
The problem
Read ›

02
User stories
Read ›

03
Proposed solution
Read ›

04
Evidence and plan
Read ›

01 · The problem

Rollback is the safety net nobody tests

Problem statement

Engineers changing payment services can't tell, before merge, whether the previous release can run safely against the state the new release creates. When it can't, a routine rollback turns an incident into a bigger one: payments are processed wrongly without any error, and some reach SWIFT, where they can't be taken back.

Where one-way doors hide

Data
Columns renamed, dropped or converted in place. Migrations edited after they ran.

Messages
Fields removed, renamed or retyped. Consumers that read a missing number as zero.

Rule data
Keys or units change in one repo while the code that reads them lives in another.

External
Payments sent to SWIFT, partner webhooks, deleted queues. Redeploying can't recall them.

Why today's checks miss it

1. **Tests run new code against new state.**
   The combination that rollback creates, old code against new state, is never exercised.
2. **Migration checks only see schemas.**
   A change to rule data or a message contract has no migration to scan.
3. **Reviewers see one repo.**
   The code that reads a changed field often lives in another team's repo.
4. **Controls fail open.**
   A missing threshold means "skip the rule", not "stop the payment".

Why payments raise the stakes

- **SWIFT has no undo.** Once sent, a payment can only be requested back through gpi Stop and Recall. After the money is credited, that needs the recipient's agreement.
- **Formats are changing now.** MT payment instructions ended on 22 November 2025. Under CBPR+, fully unstructured addresses stop being accepted in November 2026 (14 November in bank guidance).
- **Estates are split across teams.** Gateway, orchestrator, routing rules, outbound and data ingestion each have their own repo and release.

What a one-way door costs

Silent
No exception, nothing in the dead-letter queue, every dashboard green. Someone has to notice.

Irreversible
Payments at SWIFT can only be requested back. Converted or dropped data needs a restore.

Slow to unwind
Hand-written hotfixes, recall requests, incident reviews and explanations to risk and compliance.

02 · User stories

Four people, one Friday afternoon

Scenario

The rule nobody owned Northwind Payments, an invented intermediary bank



Two teams move a hold threshold to pence. The rules repo renames `holdAbove` to `holdAboveMinor` and multiplies it by 100, and the orchestrator reads the new name. Both pull requests pass. Later, the orchestrator alone is rolled back. The old code can't find `holdAbove`, skips the hold without an error, and large payments go to SWIFT unreviewed.

User stories

P **Priya**
Payments engineer · orchestrator

As a payments engineer, I want to **know before I merge whether my change can be rolled back**, so that a rollback never makes an incident worse.

T **Tom**
Rules owner · reference data

As the owner of the routing rules, I want to **see which code in other repos reads my rule data**, so that a data change never silently switches off someone else's control.

A **Aisha**
On-call engineer · reliability

As on-call, I want **a safe rollback window for every release**, so that I know whether pressing rollback will help or hurt.

G **Grace**
Head of payments operations

As head of payments operations, I want **controls like the four-eyes hold to fail closed**, so that no large payment reaches SWIFT without review.

Acceptance criteria

1. **Given** a PR that renames or rescales rule data, **when** Copilot reviews it, **then** the verdict is ONE-WAY with a safe rollback window.
2. **Given** a related repo that reads the changed key, **then** the review names the file and repo, and how many repos it checked.
3. **Given** a ONE-WAY verdict, **then** the review proposes an expand, migrate, contract, drop split, and builds it only when asked.
4. **Given** an agent editing a migration already on the default branch, **then** the edit is blocked with the reason.

Journey

Stage	Today	With One-Way Door
Review	Tests green, approved	**ONE-WAY; reader found in the other repo**
Deploy	Rules, then code, in one go	**Four safe steps across both repos**
Roll back	Hold silently skipped	**Old code still finds its threshold**
Aftermath	Large payments sent unreviewed	**Nothing leaves unreviewed**

03 · Proposed solution

A GitHub Copilot skill for the way back

One-Way Door is a **GitHub Copilot agent skill**: a folder of instructions, reference material and small scripts. Copilot loads it when someone asks to review a change, check release readiness, or asks "can we roll back?". It answers with one review comment, before merge.

What's in the box

```
.github/skills/one-way-door/
  SKILL.md · the procedure Copilot follows
  references/patterns.md · 24 one-way patterns
  references/output-format.md · the review format
  scripts/scan-migrations.mjs · risky DDL
  scripts/schema-compat.mjs · breaking events
.github/instructions/migrations…md · path rule
.github/hooks/one-way-door.json · pre-edit guard
plugins/one-way-door/ · one install, every repo
```

Sample review on PR #517

```
ONE-WAY Safe rollback window: none
closes when the rules loader runs
1. Rule parameter renamed and rescaled
   holdAbove → holdAboveMinor (×100)
   If rolled back: the old orchestrator
   skips the hold, silently.
Blast radius: RuleEvaluator.java in
   northwind-inbound-orchestrator
   not in this PR · 6 of 6 repos checked
Reply "generate the split".
```

How a review runs

1 Diff from the merge base

2 Sort files: data, messages, state, external

3 Run the scripts: offline, JSON out

4 Find readers, here and in related repos

5 Judge old code against new state

6 Write the verdict, offer the split

The safe split

1 Expand
Code reads the new key, falls back to the old, holds if neither.

2 Migrate
Rule data carries both keys. The loader checks they agree.

3 Contract
Code reads only the new key, and still fails closed.

4 Drop
Remove the old key once nothing reads it.

Design principles

- › Scripts check, Copilot judges
- › Controls fail closed
- › Says what it searched
- › Offline: git and Node 20+
- › Plain verdicts, no guesswork
- › Copilot's own formats

Hackathon build: across repos

A per-repo list of related repos, `find-readers` to search them, and data checks for renamed keys and values scaled ×10, ×100 or ×1000.

Where it runs

VS Code agent mode
Loads the skill when you ask to review or release. The path rule guides edits to migrations and contracts.

`/one-way-door · plugin or folder`

Copilot CLI
Same skill in the terminal, with the hook blocking edits to migrations already on main.

`copilot plugin install …`

Copilot cloud agent
Reviews pull requests on GitHub and posts the verdict as a comment.

`copy .github/skills/one-way-door`

04 · Evidence and plan

Proven on a real stack, ready to extend

Rollback tests, prototype

From › to	Result
main › step 1 (expand)	PASS
step 1 › step 2 (migrate)	PASS
step 2 › step 3 (contract)	PASS
step 3 › step 4 (drop)	PASS
main › the bad PR	FAIL: £0.00 rows

Each test deploys for real, sends traffic, rolls back each service in turn, and checks health, row counts, £0.00 rows, the dead-letter topic and the reconciliation total. Scenario: "The £0.00 Settlement" (Spring Boot, Kafka, Postgres).

Measured

5 / 5
incident runs reproduced from clean

61
automated tests

18 s
to reset the demo stack

24
patterns in the catalogue

Hackathon scope

- **Must:** Northwind repos (GTX, orchestrator, rules, outbound, ingestion), SWIFT and screening simulators, Spanner and BigQuery emulators, the story end to end, multi-repo skill support.
- **Stretch:** the structured-address scenario, an approve endpoint for held payments, the recall flow.

Success measures (targets)

- The review on PR #517 says ONE-WAY and names RuleEvaluator.java.
- Cross-repo rollback tests: four steps pass, the bad change fails.
- No TWO-WAY verdict on any change the tests show is unsafe.

Risks and limits

- Spanner and BigQuery run as emulators, so behaviour can differ.
- pacs.008 is a subset; no settlement or account posting.
- Copilot decides to load a skill from its description. A manual checklist covers each surface.
- Judgment can be wrong, so every review states what it searched and what it couldn't reach.

Roadmap

Now: built
- Skill, catalogue, two scripts
- Path rule and pre-edit hook
- Copilot plugin, rollback proof

Next: this hackathon
- Readers across repos
- Rule-data checks
- Northwind SWIFT and GTX demo

Later
- Every PR via Copilot code review
- Spanner and BigQuery checks
- CHAPS and Faster Payments

The ask

Try it on the repos that hold migrations, message contracts or rule data, and tell us where it missed a door. Contact: [Team name] · [Contact].

Try it on your repos ›